01 / OBJECTIVE AND RECOMMENDATION

SubsampWinner Implementation Refactoring Proposal

For review • October 17, 2026 • Python baseline e0366d3

Primary objective
Make the existing scientific implementation readable, maintainable, testable, and reliable, while preserving its core algorithm and valid numerical behavior.
Create a clear foundation for future algorithm improvements and features.

Use an incremental refactor of the existing classes and calculations, with small, reviewable changes protected by reference tests. A wholesale rewrite would introduce avoidable numerical and compatibility risk. AI assistance supports this engineering work; it does not change the package's scientific purpose.

Current foundation

The package uses NumPy, statsmodels, and SciPy; the inspected source has no Torch dependency. Keep that stack and statsmodels' OLS/WLS fits in this phase. The main work is clearer responsibilities, explicit contracts, isolated randomness, and less duplicated state and storage.

Stage 1 has already established saved data and feature-subset schedules, independent OLS/WLS checks, defect reproductions, and a comparison with archived CRAN code. The recorded local run passed 15 tests, including the R comparison [4]. Those results provide a baseline, not a claim that all current behavior is correct.

Treatment of the supplied review [5]

Recommendation	Treatment in this proposal
Reference behavior first	Adopted and already substantially completed; close the remaining targeted coverage gaps before extracting the core.
Clear stages, compact statistics, optional MPI	Adopt as implementation improvements, with equivalence checks at each stage.
Union-refit-select assurance, new selection rules	Defer as method changes. Preserve the existing adaptive procedure and document its limitations.
Simulation study and release work	Separate later work. This phase uses small local checks and benchmarks; it includes no publication or release.

This document proposes the next refactoring work for evaluation. It does not implement that work or change the package's defaults.

02 / PROPOSAL FOR REVIEW

Behavior to preserve

The preservation target is the current Python algorithm on its valid, documented input domain. The paper and R code explain the method and expose discrepancies; they do not automatically replace existing Python choices [1-4].

Part of the workflow	Refactoring invariant
Feature sampling	Draw s distinct columns per repetition, use all observations, and perform m repetitions. Repeated subsets across repetitions remain possible.
Base regression	Preserve OLS or user-weighted WLS with an intercept. Preserve conventional model statistics and original feature identities.
Scoring	Average abs(t) / sqrt(RSS) over retained nonzero t statistics. Preserve zero scores when a feature has no nonzero contribution.
Retention and ranking	Keep use_top_s=False as the existing default; True retains the best min(s,m) submodels. Keep qnum=None meaning s.
Final filtering	Preserve Bonferroni by default, BH when requested, and an explicit unadjusted option. The adjustment family remains the q semifinalists.
Result interpretation	final_model is the joint semifinalist model; finalists are a significance-filtered index list. Do not refit automatically on finalists.
Adaptive assurance	Preserve its range formula, intersection/union score, largest-set update, and cumulative union on valid nonempty cases. No union-model refit is added.

What the reference comparison established

The accessible preprint's equation (3) uses 1/sqrt(RSS). The Python computation agrees; its 1/RSS docstring is incorrect. The R scorer uses residual standard error, sqrt(RSS/d), so its scores equal sqrt(d) times Python's top-s scores when every OLS submodel has the same d = n - s - 1 [1,2].

The saved example gave a maximum difference of 4.44e-16 after this scale adjustment. R's final stepwise AIC selection differs from Python's significance filter, so agreement in finalists is not an acceptance criterion. The final journal equation is still unverified because full-text access failed [3,4].

Changes that require a separate method decision

Changing the all-model default, the RSS exponent, the nonzero-statistic count, the multiplicity family, the WLS scoring criterion, or the assurance selection rule is outside the refactor. WLS currently scores raw, unweighted RSS; preserve that adaptation while documenting it. Do not add an RSS floor, silently resample failed fits, or silently drop predictors.

03 / PROPOSAL FOR REVIEW

A smaller, clearer implementation

Keep the existing import paths and public class names. Make subsamp a thin coordinator over small typed helpers; keep SubsampDoubleAssurance as composition over subsamp. Organize by statistical responsibility and extract private modules only when reuse or clarity warrants it.

Responsibility	Proposed boundary and purpose
Validate inputs	Check dimensions, finite values, counts, weights, and option names before fitting. Share these checks between entry points.
Schedule subsets	Produce ordered feature subsets from an explicit local random stream. Accept a saved schedule internally for numerical comparisons.
Analyze one subset	Fit one specified design with statsmodels; return feature IDs, t statistics, raw RSS, and residual degrees of freedom.
Aggregate scores	Choose retained repetitions, accumulate scores and nonzero counts, then rank features. Keep sampling and fitting out of this helper.
Fit and filter	Fit semifinalists once, align p values with feature IDs, and apply the chosen adjustment. Reuse the same base fitting helper.
Evaluate assurance	Calculate set union/intersection and the current update criterion separately from iteration control and printing.

Class and result discipline

Compute intermediate values locally and assign fitted attributes together after success, so an exception cannot leave a mixture of old and new results. Retain feature_weights, semifinalists, finalists, p_adjusted, final_model, X, and y with explicit meanings. Preserve the historical subsample_anlysis spelling through a compatibility wrapper if an internal corrected name is introduced.

Use at most one small private record for submodel statistics if it clarifies the data flow. Keep compatibility adapters for methods that currently return dictionaries or dense vectors. Avoid a plugin framework, estimator hierarchy, or new branded public API.

Optimize storage after equivalence is established

Internally store each submodel's s feature IDs and s statistics, rather than a length-p vector. This changes submodel storage from O(mp) to O(ms), plus O(p) score accumulators; it is a storage analysis, not a measured speedup. Preserve repetition order during summation and retain explicit zero statistics for the denominator rule.

Keep the regression solver unchanged. Profile small local cases before attempting solver batching, alternate linear algebra, or additional parallel backends. Torch, GPU support, and new numerical dependencies are not needed for this refactor.

04 / PROPOSAL FOR REVIEW

Discussion and compatibility choices

These are proposed policies for review, not decisions already implemented. They repair undefined or misleading behavior while leaving the valid numerical workflow intact.

Issue	Recommended treatment and trade-off
Invalid inputs and qnum=0	Reject invalid counts, shapes, nonfinite values, and unknown method names with actionable errors. Treat qnum=0 as invalid; empty finalists remain a valid outcome after filtering.
Constants and singular fits	Reject supplied constant predictors and nonidentifiable fitted designs; construct the intercept explicitly. Check rank on each fitted subset and the semifinalist fit, not on the entire high-dimensional X. Do not silently remove or resample columns.
Saturated or perfect fits	Require positive residual degrees of freedom and finite model statistics. Fail explicitly for zero RSS. Any near-zero tolerance must be specified and tested rather than becoming a hidden score adjustment.
Random-state compatibility	First isolate the legacy sampler in a local RandomState seeded per fit, preserving historical serial sequences where verified. A later Generator migration is a separate reproducibility choice. seed=None should use local entropy, not the caller's global NumPy stream.
fit() return value	Preserve None for compatibility in this phase and correct the documentation. Returning self is reasonable later, but is an API decision rather than a prerequisite for maintainable internals.
Assurance boundary cases	Return a documented empty result when all selections are empty, without calling it perfect stability. Validate candidate sizes and reject infeasible triplets; clipping or resampling would need a separate rule. Report the actual last evaluated range.
Simulation covariance	Keep simulation utilities separate from algorithm verification. Add validity checks now; choose a PSD-generating recipe in a separately reviewed simulator repair because it changes the data-generating process.

Parallel result contract

For a future MPI repair in this phase, recommend one schedule created on root, jobs identified by repetition, canonical aggregation order, and the complete fitted result on every rank. This is easiest to reason about, but broadcasts add memory and communication cost. Root-only results are an alternative that requires a clearly documented worker return contract.

A rank-local failure must be reported collectively before a coordinated failure; otherwise other ranks can hang. Actual multi-process tests are required before describing MPI support as repaired.

05 / PROPOSAL FOR REVIEW

Execution sequence and acceptance

Change set	Deliverable and acceptance condition
1. Complete the guardrails	Add the supplied hand-calculated retention fixture, an analytical orthogonal OLS example with nonzero intercept, and focused repair tests. Keep defect observations separate from preservation fixtures.
2. Extract the serial core	Introduce the helper boundaries and simplify class state, keeping the solver and defaults unchanged. Replay every saved OLS/WLS schedule; compare fits, scores, rankings, adjusted p values, and finalists.
3. Repair contracts	Implement the reviewed validation, intercept, RNG, and empty-result policies. Each changed behavior gets a focused test, rationale, and compatibility note. Preserve valid historical outputs.
4. Reduce duplication	Introduce compact internal statistics and shared fitting logic. Recheck arithmetic and ordering, then measure runtime and peak memory on bounded local workloads.
5. Isolate parallel execution	Make MPI lazy and optional. Add schedule-based worker execution and collective error handling only after the serial core passes. Validate multiple ranks, uneven allocation, and idle ranks.
6. Review the result	Deliver the local diff, updated behavior documentation, validation commands/environment, and measured limitations. Limit metadata edits to optional MPI and an honest supported Python range.

Required evidence

Use four layers: historical characterization, independent arithmetic, fixed-schedule R/Python comparison, and contract/failure tests. Compare R/Python only where their procedures match; correct the known score scale. Keep current reference tolerances unless conditioning or changed evaluation order justifies a documented adjustment.

Check nonzero-count semantics, feature-ID alignment, valid empty finalists, global RNG isolation, and predictor-rescaling/response-shift invariance with fixed schedules. A serial import and fit must work with MPI absent. MPI equivalence should compare all ranks against the same serial schedule, not unrelated seeds.

Bounded optimization and delivery

Use representative local cases such as (n,p,s,m) = (80,100,10,100) and (200,1000,20,200); record hardware, library versions, runtime, and peak memory. These are proposed benchmarks, not completed measurements. No speedup target is asserted before profiling.

The existing 15-test run is a recorded stage-1 result, not evidence that these proposed repairs already pass. Add local test execution now; a repository test-CI change can be reviewed later without modifying the publishing workflow or remote settings. No HPC runs or broad simulation study are needed to accept the refactor.

06 / PROPOSAL FOR REVIEW

Evaluation and evidence

Questions for review

Decision	Recommended answer
What is being preserved?	The valid current Python workflow, including its documented adaptations and defaults. The reference sources explain differences, rather than silently overriding them.
What changes now?	Internal structure, data flow, documentation, numerical contracts, randomness ownership, and avoidable storage. Fixes to invalid cases are explicit.
What makes it ready?	A readable orchestration path, independently checked helpers, equivalent valid outputs, clear failure behavior, and reproducible local evidence.
What remains separate?	New scoring or inference rules, union-refit-select assurance, new public APIs, Torch/GPU work, new R implementation, rebranding, release, and publication.

Recommended next authorization
Proceed with the serial refactor and explicit contract repairs after reviewing the policies on page 4. Treat compact storage and MPI as subsequent, separately reviewable changes within the same implementation objective.

Sources and provenance

[1] Fan, Y. and Sun, J. (2020). *Subsampling Winner Algorithm for Feature Selection in Large Regression Data.* arXiv:2002.02903v1. Algorithm 1, equation (3), Algorithm 3. Primary preprint.

[2] CRAN *subsamp* 0.1.0 (2017), regression implementation. Pinned mirror e56d150; the archived source was inspected and executed locally. R source | CRAN archive.

[3] Fan, Y. and Sun, J. (2021). *Subsampling from features in large regression to find winning features.* Statistical Analysis and Data Mining 14(2), 168-184. doi:10.1002/sam.11499. Bibliographic record verified; final full-text equation not verified.

[4] Local evidence: `docs/REFERENCE_BEHAVIOR.md`, `docs/REFERENCE_HANDOFF.md`, and `tests/reference/`. Python baseline e0366d3. Recorded environment: Python 3.10.7, NumPy 1.26.4, SciPy 1.13.0, statsmodels 0.14.2, mpi4py 4.1.1, pytest 8.4.2; R 4.3.2. MPI initialization disabled for serial tests.

[5] User-supplied review memo, beginning "I recommend making the first milestone establish and verify reference behavior," supplied October 17, 2026. Its prior source-only assessment is incorporated and updated with the completed local reference work. The published-PDF check identified in that memo remains outstanding.

Scientific limits

Implementation equivalence does not establish general FDR control, post-selection validity, heteroskedastic robustness, or large-scale performance. The full journal audit and actual MPI execution remain open. Future algorithm extensions should use this refactored core as their comparison baseline and receive their own statistical evaluation.